

University of Stuttgart
Germany

Internet of Things

Ilche Georgievski
ilche.georgievski@iaas.uni-stuttgart.de

Summer

Sensors

- Founding blocks
- Core functions
 - Collect the state of sensed entity
 - Send collected data to a gateway

Actuators

- Founding blocks
- Core functions
 - Receive commands
 - Execute actions on entities

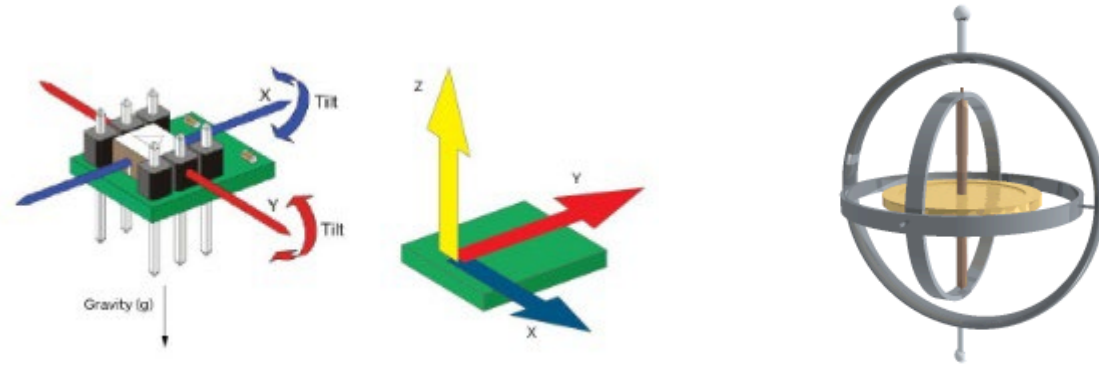
Types of IoT

- Electronic-based IoT
- Software-based IoT
- Human-based IoT
- Simulated IoT
- Virtual IoT

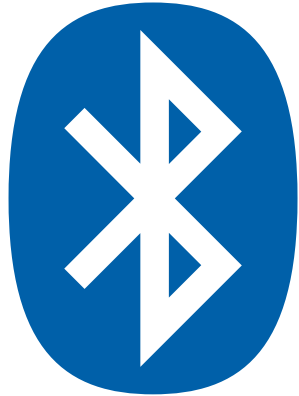
ELECTRONIC-BASED IOT

Electronic-based IoT

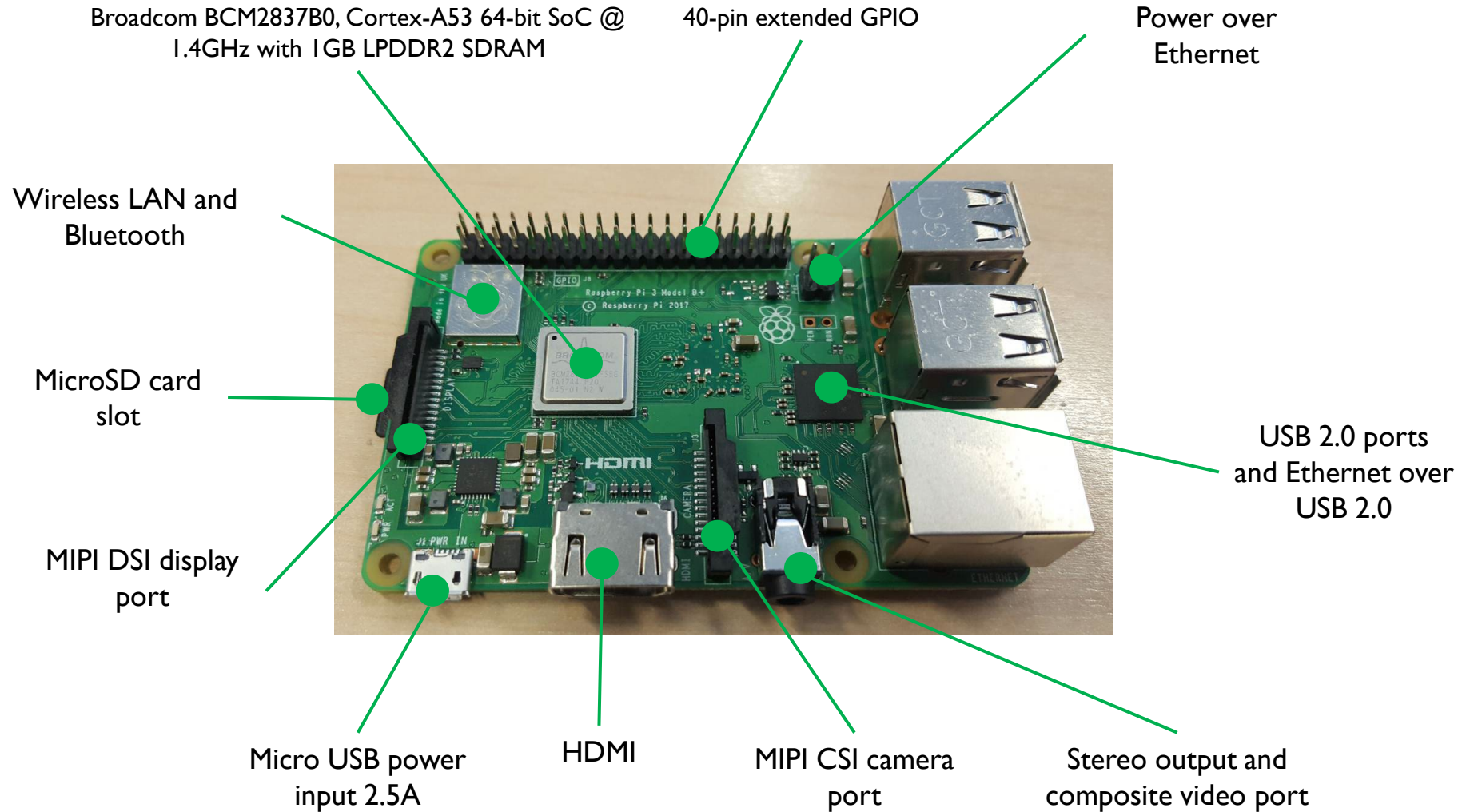
- Personal devices
 - Smartphones
 - Smart watches
 - Fitness trackers
- Special-purpose devices
 - Measure and report specific conditions (e.g., light, temperature)
 - Control (e.g., turn valves, switch lights)



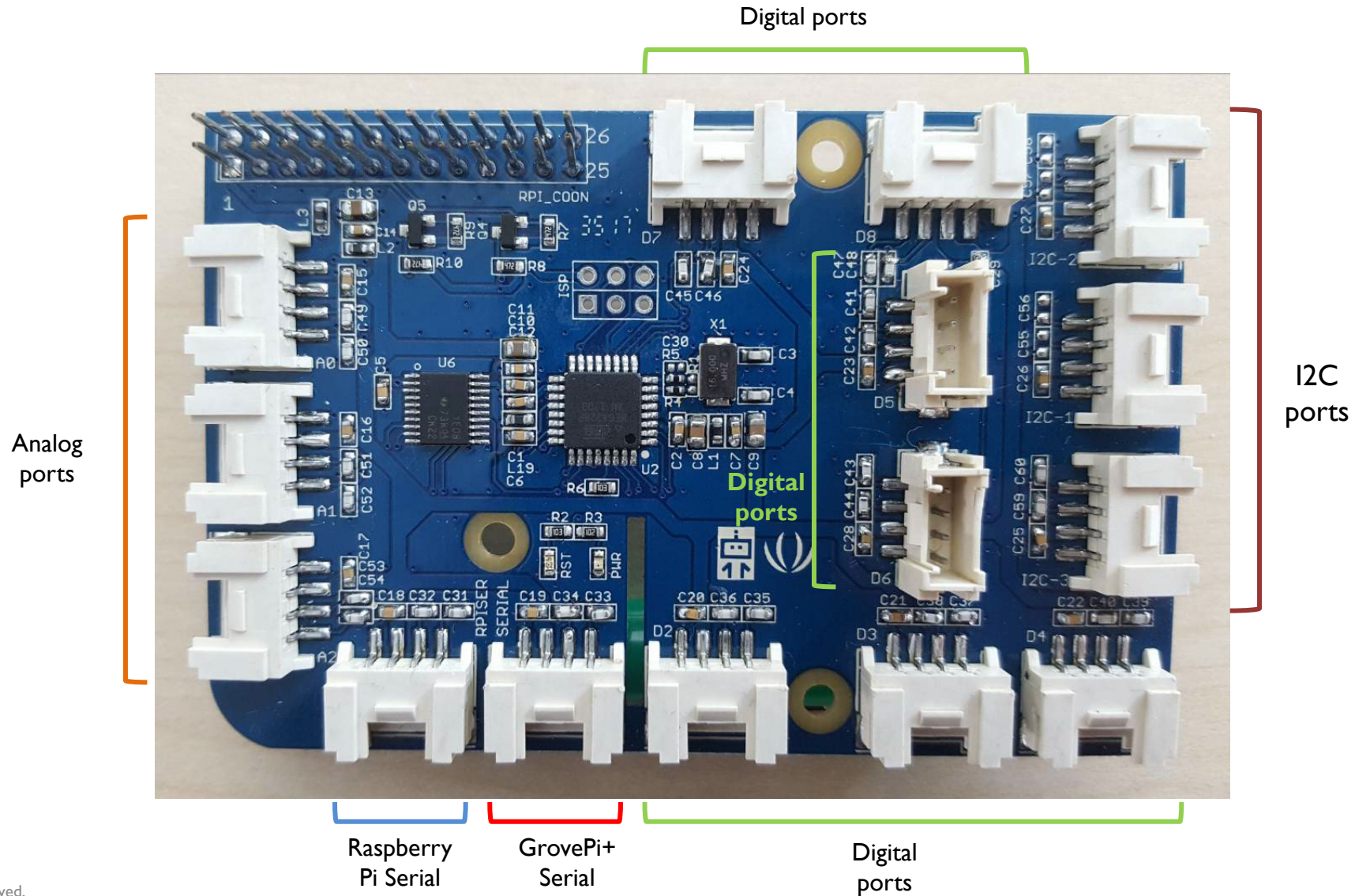
Protocols

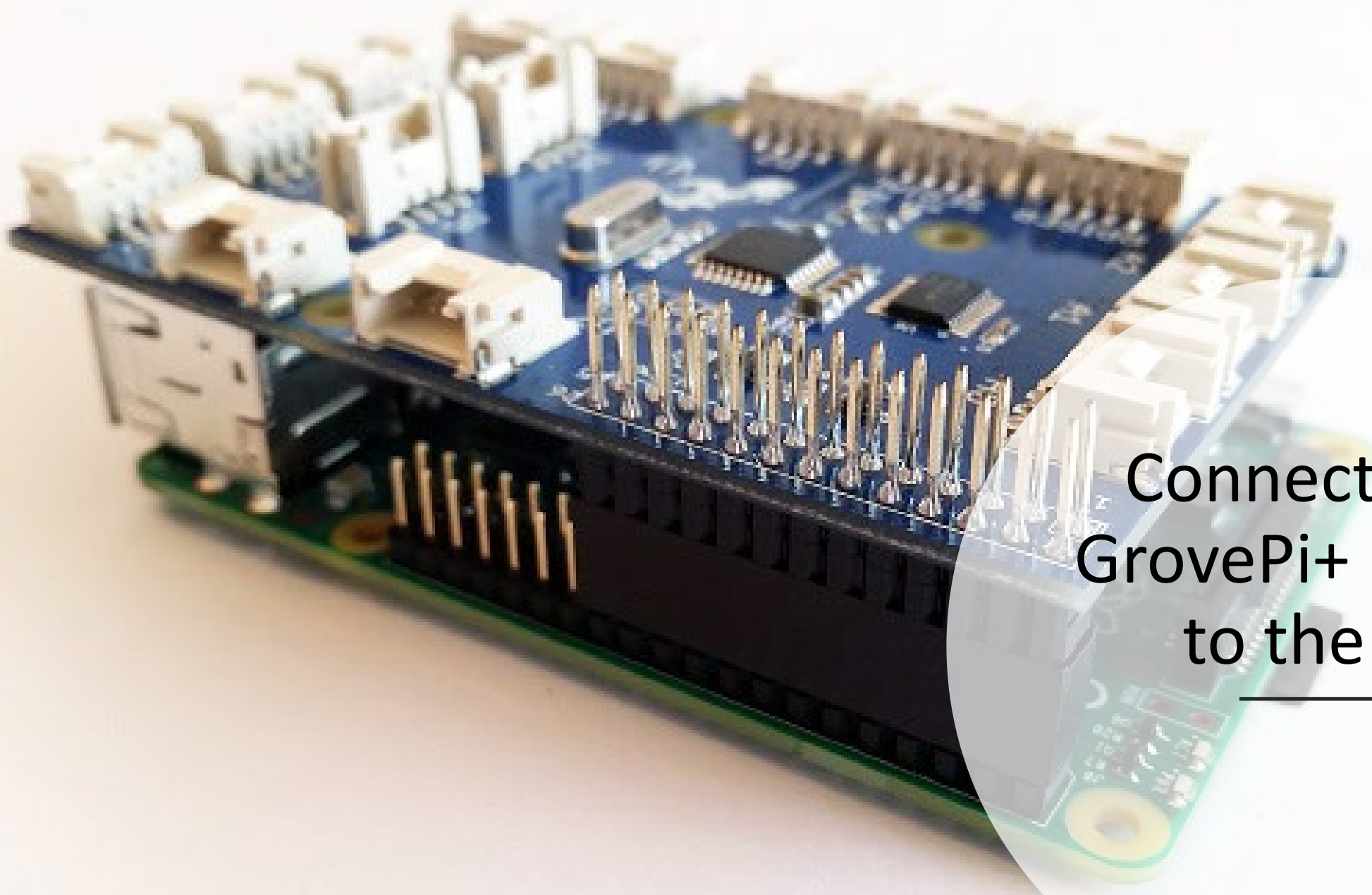


Raspberry Pi

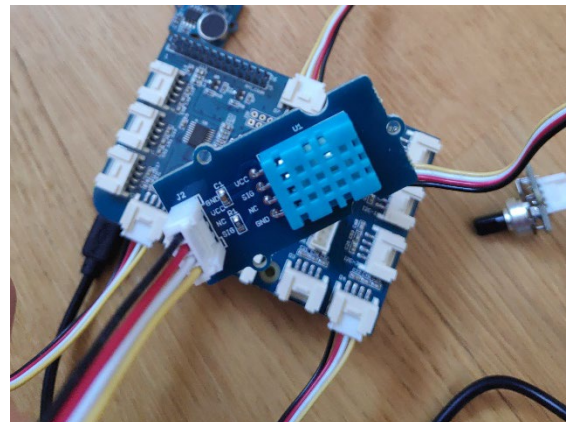
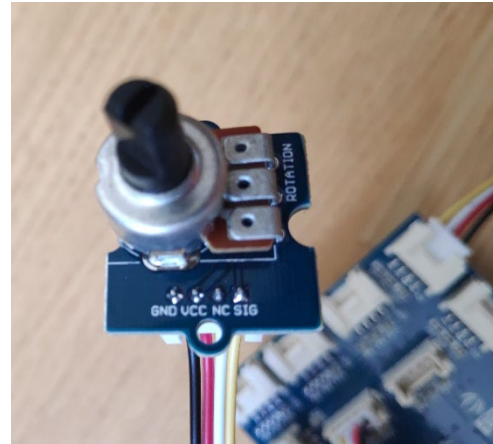
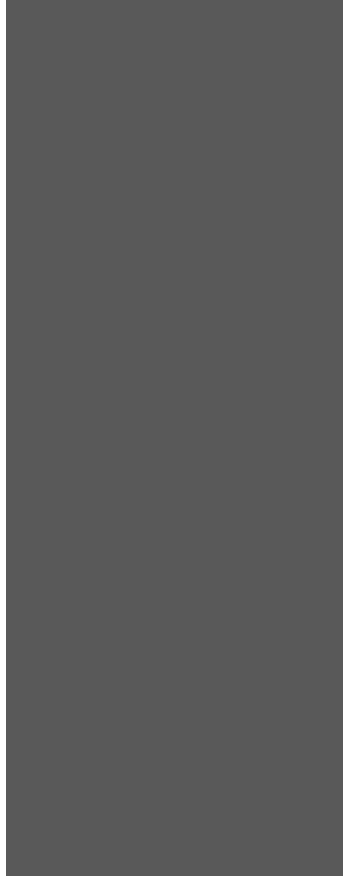
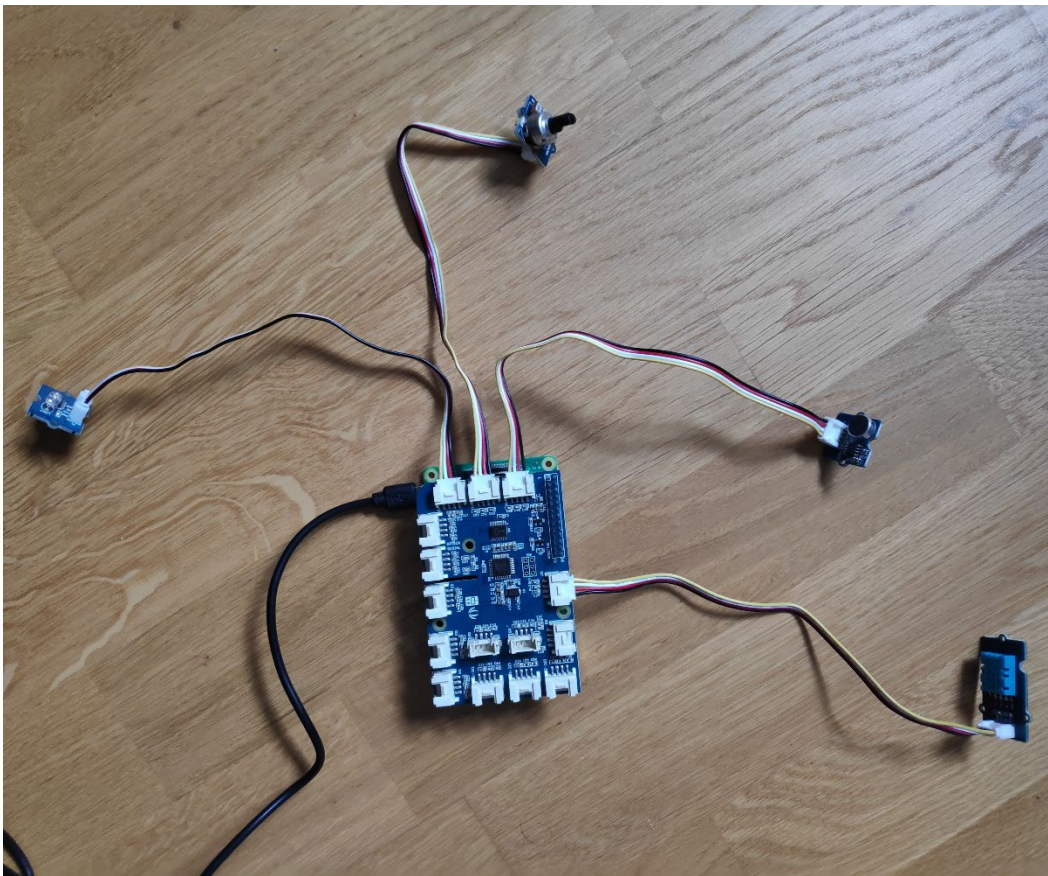
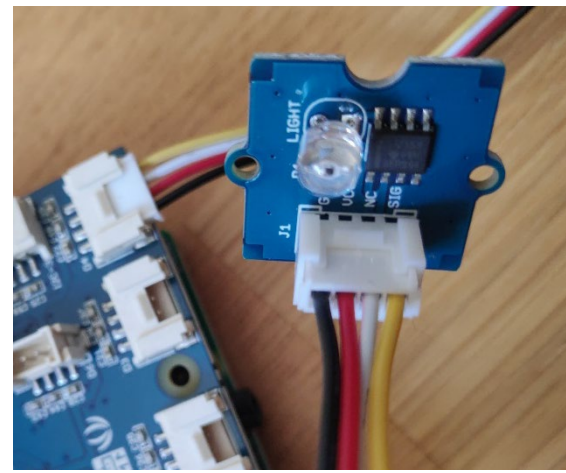
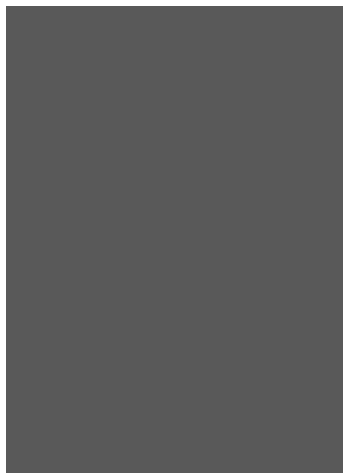
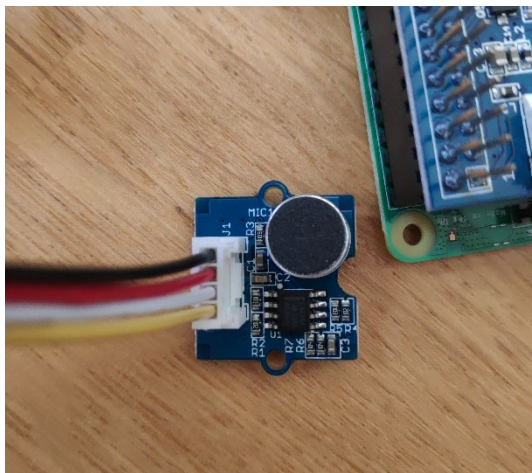


GrovePi Board



The image shows a GrovePi+ board, which is a blue PCB populated with various electronic components like resistors, capacitors, and integrated circuits. It features a large array of gold-plated pins on its underside. This board is being connected to a Raspberry Pi, which is a green PCB with its own set of pins visible at the bottom. A black plastic connector is used to bridge the pins of the two boards. A semi-transparent circular graphic is overlaid on the right side of the image, containing the text.

Connect the
GrovePi+ board
to the Pi



Read Grove temperature and humidity sensor

```
import grovepi  
import time
```

```
port=7  
sensor=0  
timeout=1
```

```
while True:  
    [temperature,humidity] = grovepi.dht(port,sensor)  
    print("{} Temperature = {} C, humidity = {} %".format(time.ctime(),temperature,humidity))  
    time.sleep(timeout)
```

Aeotec MultiSensor6

- **Capability**
 - Reports motion, temperature, humidity, luminance, ultraviolet level, and vibration
- **Features**
 - Routing binary devices based on the Z-Wave protocol
 - Supports Security Command Class with AES-128-bit security encryption
- **Operating distance**
 - Up to 150 meters outdoors
- **Operating temperature**
 - 0°C to 40°C
- **Relative humidity**
 - 8% to 80%



Aeotec Z-Stick

- **Capabilities**

- Z-Wave USB adapter for creating a network
- Remotely adds/removes Z-Wave devices

- **Compatibility**

- Host controller can be a part of the network
- Works with other Z-Wave devices from other manufacturers

- **Operating distance**

- Up to 150 meters outdoors



Read Aeotec MultiSensor6

```
from openzwave.option import ZWaveOption
from openzwave.network import ZWaveNetwork
from openzwave.node import ZWaveNode
from openzwave.value import ZWaveValue
from time import sleep
from pprint import pprint
```

```
options = ZWaveOption("/dev/ttyACM0", config_path='/home/pi/Downloads/python-openzwave/openzwave/config', user_path=".")
options.set_console_output(False)
options.set_logging(False)
options.set_append_log_file(False)
options.set_save_log_level('Error')
options.lock()
```

```
network = ZWaveNetwork(options, log=None)
network.start()

while network.state != network.STATE_READY:
    sleep(1)

print("Network is ready")
```

```
for node in network.nodes:
    for sensor in network.nodes[node].get_sensors().items():
        print("Sensor type: '{0}'".format(sensor[1].label))
        print("Sensor reading: '{0}'".format(sensor[1].data))
```

Network controller

- Controls and organises the network
- Communicates with other nodes of the mesh network

Main controller

- Connection between the ZigBee network and the server
- Uses a serial protocol
- Communicates directly to the Network controller



Circle+

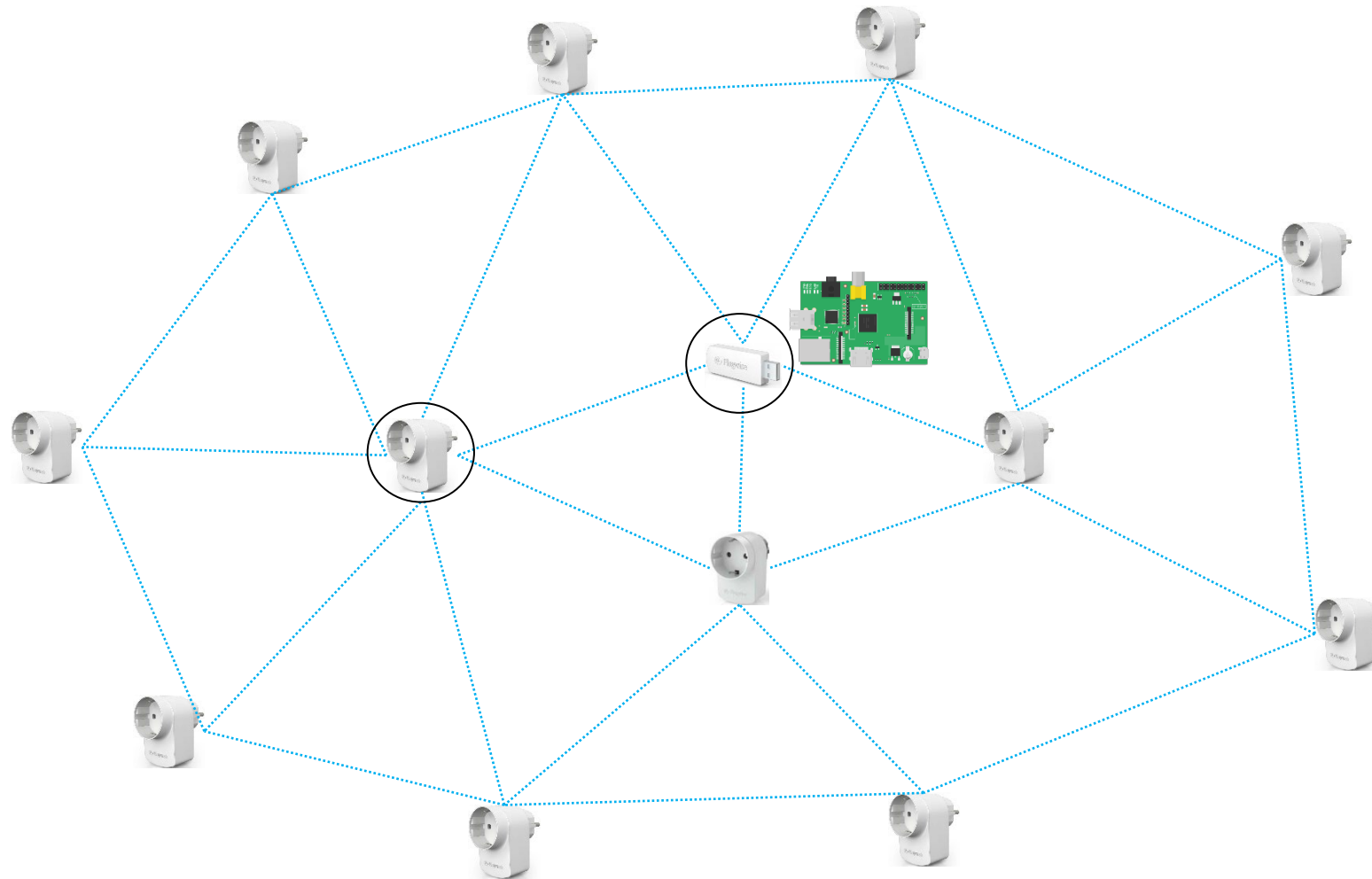


Circle



Stick

Zigbee Mesh Network



Read/Control Plugwise actuator

```
from plugwise.api import *  
from time import sleep  
from pprint import pprint
```

```
DEFAULT_PORT = "/dev/ttyUSB0"  
mac = "000D6F0005A91442"
```

```
stick = Stick(DEFAULT_PORT)  
circle = Circle (mac, stick)
```

```
print("Current electricity consumption in W: %.2f" % (circle.get_power_usage(),))
```

```
circle.switch_off()  
print("Info:")
```

```
if circle.get_info()['relay_state'] == 0:  
    print("The Circle has been turned off successfully")  
else:  
    print("Turning off the Circle did not quite work")
```

```
sleep(10)
```

```
circle.switch_on()  
print("Info:") pprint(circle.get_info())
```

Kiebeck&Peter

- **en:key room thermostat**
 - Solar powered and self-learning thermostat with LCD
 - Temperature sensor
 - Occupancy sensor
 - Room controller
- **EnOcean valve actuator**
 - Wireless and self-powered actuator
 - Built-in temperature measurement



Parsing EnOcean

```
val data = packet.data.slice(1, packet.data.size).toArray
val currentValue = ByteUtils.subDataBits(0, 8, data)(0)
val serviceOn = ByteUtils.subDataBits(8, 1, data)(0) != 0
val energyInputEnabled = ByteUtils.subDataBits(9, 1, data)(0) != 0
val energyStorage = ByteUtils.subDataBits(10, 1, data)(0) != 0
val batteryCapacity = ByteUtils.subDataBits(11, 1, data)(0) != 0
val coverOpen = ByteUtils.subDataBits(12, 1, data)(0) != 0
val failureTempSensor = ByteUtils.subDataBits(13, 1, data)(0) != 0
val windowOpenDetection = ByteUtils.subDataBits(14, 1, data)(0) != 0
val actuatorObstructed = ByteUtils.subDataBits(15, 1, data)(0) != 0
val temperature = ScalaByteUtils.byteToInt(ByteUtils.subDataBits(16, 8, data)(0)) * 40 / 255
val IrnBit = ByteUtils.subDataBits(28, 1, data)(0) != 0

val message = ActuatorMessage(currentValue, ... , IrnBit)
```

Tutorials

- Raspberry Pi
- GrovePi+
- Aeotec
- Plugwise

SOFTWARE-BASED IOT

Software-based IoT

- Operates entirely in the digital/information world
- Software:
 - Senses digital information
 - Acts on digital information
- Two classes
 - Custom software solutions
 - Existing Web services

Custom software-based sensors

- Detect computer power state
- Detect computer activity
 - Active, inactive, idle (e.g., native Windows function `GetLastInputInfo`)
- Detect human presence
 - Wi-Fi probing
 - [Probemon](#): Monitor and log 802.11 probe request frames
 - [Airodump-ng](#): packet capture, capturing raw 802.11 frames.
- ...

Some existing Web services as sensors

- Weather forecast
 - Open Weather (<https://openweathermap.org/api>)
 - Weatherbit (<https://www.weatherbit.io/api>)
 - Tomorrow (<https://www.tomorrow.io/weather-api/>)
- Environmental data
 - BreezoMeter Air Quality API (<https://www.breezometer.com/air-quality-api/>)
 - CO2 Signal API (<https://co2signal.com/>)
- Electricity (consumption, generation, market data)
 - SMARD (<https://www.smard.de/en/downloadcenter/download-market-data/>)
 - ENTSOE (<https://transparencyplatform.zendesk.com/hc/en-us/articles/15692855254548-Sitemap-for-Restful-API-Integration>)
 - ElectricityMap API (<https://www.electricitymaps.com/>)
- Calendar
 - Google Calendar API (<https://developers.google.com/calendar/>)

Software-based actuators

- Set clock of an alarm
- Trigger a reminder
- Make an appointment
- Send notification (e.g., SMS, email, messaging app)
- ...

HUMAN-BASED IOT

Human-based IoT

- Humans act as sensors and/or actuators within an IoT system
- Sensing is performed through human perception, interpretation, and reporting
- Actuation is performed through humans performing actions
- Enables interaction with aspects that are difficult or impossible to capture automatically

Humans as sensors

- What is the temperature level at your location?
 - Cold
 - Comfortable
 - Hot
- How full is the waste bin at your location?
 - Empty
 - Somewhere in the middle
 - Full

Humans as sensors

- What are you currently busying with?
 - Writing
 - Reading
 - Studying
 - Meeting
 - Eating
- Is there sufficient light for the type of activity you currently perform?
 - Yes
 - No

Humans as actuators

Operation

- Turn on light
- Open window
- Pull down window blinds
- ...

Assistance

- Take medicine
- Wash hands
- Take a break
- Sit straight
- Ride bike
- ...



© Fraunhofer

How to empower humans?

- Dedicated display
- Personal devices



- User interface
 - Web-based interface
 - Push notifications (e.g., <https://web.dev/explore/notifications>)
 - Bot
 - Telegram Bot (<https://core.telegram.org/bots/api>)
 - Dedicated dialogue app

Temperature variation

- Physical temperature sensor
- Software-based sensor
- Human-based sensor

SIMULATED IOT

Simulated IoT

- Sensing and/or actuation are artificially generated
- Sensors produce synthetic data based on models, rules, or predefined scenarios
- Actuators simulate effects or responses without impacting real systems
- Used when real sensing/actuation is unavailable, impractical, or undesirable
- Enables controlled, repeatable, and scalable scenarios
- Examples
 - Simulated traffic data
 - Emulated device behavior in large-scale IoT experiments
 - What-if scenario generation for planning and validation

Options

- Custom implementation
- IoT Device Simulation platforms

Considerations

- Device model
 - Device type
 - Unit
 - Expected value range
 - Minimum value
 - Maximum value
 - Average value
 - Value variation
 - Value generation
 - Increase/decrease
 - Randomised
 - Based on another input
- Simulation
 - Number of simulated devices (per type)
 - Sensing interval
- Simulation of data noise, device failures and network disruptions

Temperature sensor model

- **Device type:** temperature
- **Unit:** Celsius
- Expected value range
 - **Minimum value:** 16
 - **Maximum value:** 35
- **Average temperature:** 20
- **Temperature variation:** 20%
- Value generation
 - **Increase/decrease:** 0.5 degrees
 - **Randomised:** based on min/max values or based on average and min/max
 - **Based on another input:** Web service on current weather data

Simulation

- **Number of devices**
 - Two types: temperature and humidity sensors
 - 30 instances of each sensor
- **Sensing interval**
 - 5 seconds for the temperature sensor
 - 15 seconds for the humidity sensor

Implementation

- Minimal set of functions for sensors
 - Initialisation
 - Sensing
- Minimal set of functions for actuators
 - Initialisation
 - Actions

```
from random import random
```

```
class TemperatureSensor:
```

```
    sensorType = "temperature"
```

```
    instanceID = "32kd403ks"
```

```
    unit = "celsius"
```

```
    def __init__(self, averageTemperature, temperatureVariation, minTemperature, maxTemperature):
```

```
        self.averageTemperature = averageTemperature
```

```
        self.temperatureVariation = temperatureVariation
```

```
        self.minTemperature = minTemperature
```

```
        self.maxTemperature = maxTemperature
```

```
        self.value = 0.0
```

```
    def sense(self):
```

```
        # self.value = self.value + self.simpleRandom()
```

```
        self.value = self.complexRandom()
```

```
        return self.value
```

```
    def simpleRandom(self):
```

```
        value = self.minTemperature + (random() * ((self.maxTemperature - self.minTemperature)))
```

```
        return value
```

```
    def complexRandom(self):
```

```
        value = self.averageTemperature * (1 + ((self.temperatureVariation / 100) * (3 * random() - 1)))
```

```
        value = max(value, self.minTemperature)
```

```
        value = min(value, self.maxTemperature)
```

```
        return value
```

```
import time
from SimulatedIoT import TemperatureSensor
```

```
class Simulator:
    def __init__(self, interval):
        self.interval = interval

    def start(self):
        ts = TemperatureSensor(20, 10, 16, 35)
        while True:
            print(ts.sensorType, ts.sense())
            time.sleep(self.interval)
```

```
s = Simulator(5)
s.start()
```

IoT Device Simulation Platforms

Paid platforms

[AWS IoT Device Simulator](#)

[IoTIFY](#)

[Bevywise IoT Simulator](#)

[IBM Watson IoT Simulator](#)

Key features

Simulate thousands of devices, AWS integration

Cloud-based, scalable, supports MQTT/HTTP/CoAP

Lightweight MQTT simulation, customisable

Simple device emulation, IBM Cloud integration

Notes

Pay-per-use on AWS

Low-cost subscription plans

One-time license fee

Free tier available

Open-source platforms

[Azure IoT Device Simulation](#)

[ThingsBoard Emulator](#)

[Kaa IoT Platform](#)

[Golioth Device Simulator](#)

Key features

Scale testing, integrates with Azure IoT Hub

MQTT, CoAP, HTTP device simulation

Full IoT platform with device simulation support

Cloud-native, Zephyr OS support

Notes

Free, but Azure services cost

Fully free, customisable

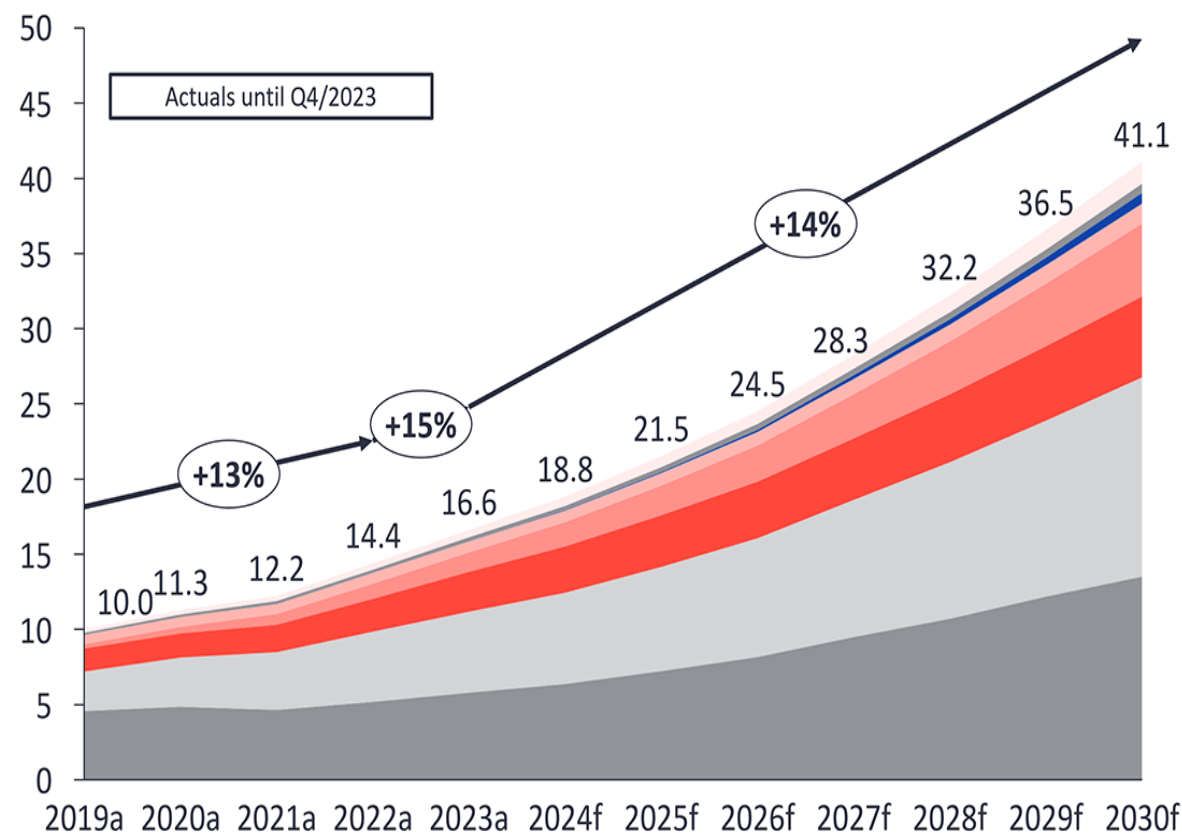
Free core, enterprise paid

Free for use

VIRTUAL IOT

Global IoT market forecast (in billions of connected IoT devices)

Number of global active IoT connections (installed base) in billions



Connectivity type	CAGR 21–23	CAGR 23–30
Other	21%	17%
Wireless neighborhood area networks (WWAN)	15%	14%
Cellular 5G IoT	147%	62%
Wired IoT	4%	9%
LPWA	35%	21%
Cellular IoT (excl. 5G, LPWA)	21%	11%
Wireless local area networks (WLAN)	18%	14%
Wireless personal area networks (WPAN)	12%	13%

XX% = CAGR

Note: IoT connections do not include any computers, laptops, fixed phones, cellphones, or consumers tablets. Counted are active nodes/devices or gateways that concentrate the end-sensors, not every sensor/actuator. Simple one-directional communications technology not considered (e.g., RFID, NFC). Wired includes ethernet and fieldbuses (e.g., connected industrial PLCs or I/O modules); Cellular includes 2G, 3G, 4G, 5G; LPWA includes unlicensed and licensed low-power networks; WPAN includes Bluetooth, Zigbee, Z-Wave or similar; WLAN includes Wi-Fi and related protocols; WWAN includes non-short-range mesh, such as Wi-SUN; Other includes satellite and unclassified proprietary networks with any range.

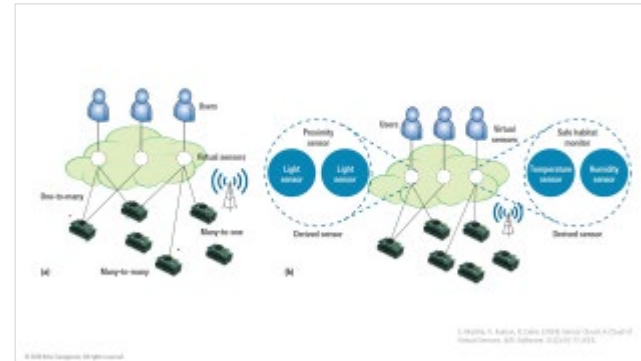
Source: IoT Analytics Research 2024-State of IoT Summer 2024. We welcome resharing: Please attribute this image to its original source and include a link back to the original article.

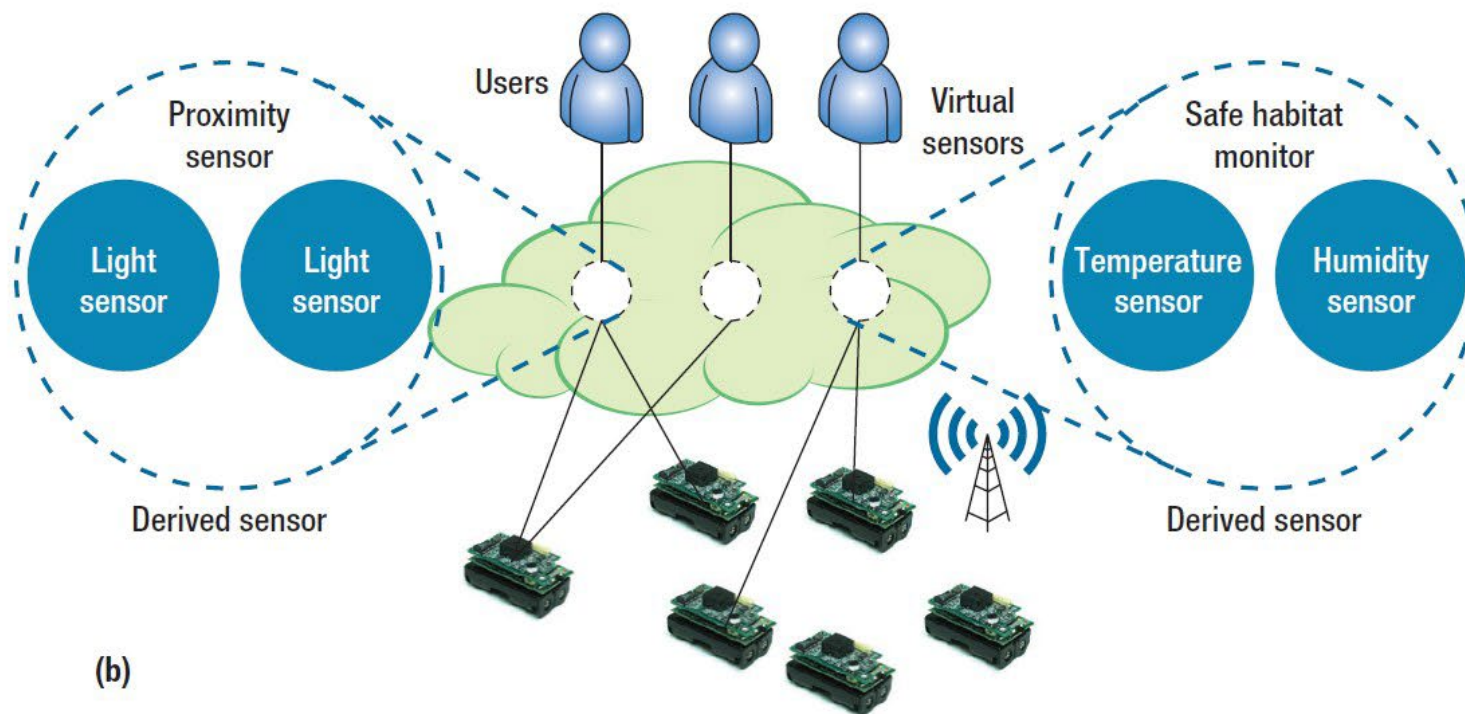
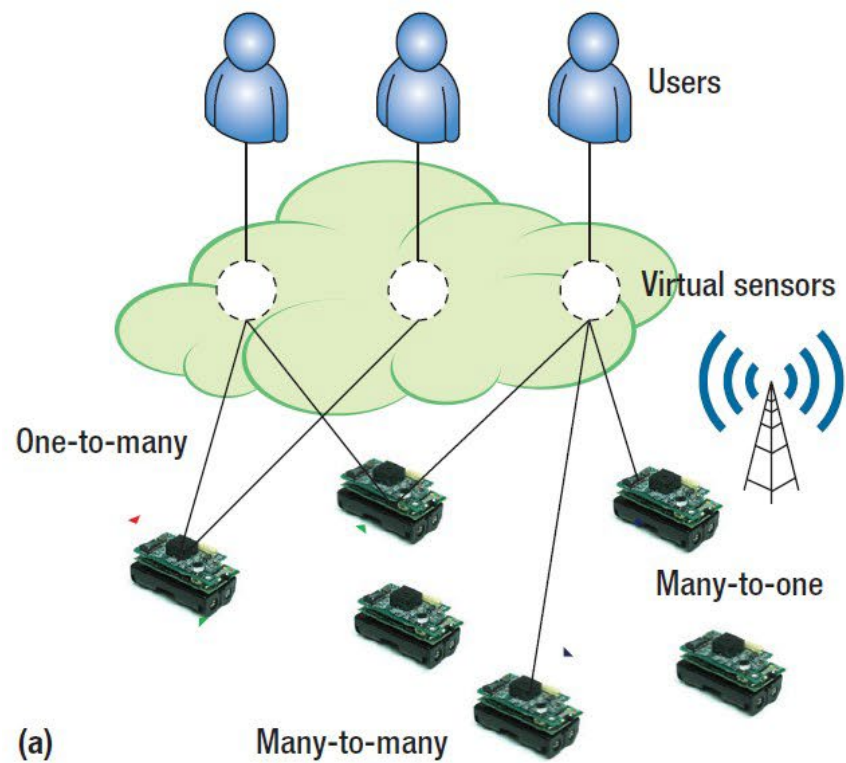
Virtual IoT

- New IoT entities are created from data produced by existing IoT
- Sensing and/or actuation are derived, not directly observed
- Virtual “things” represent computed or inferred states of the system
- Can represent aspects that are not directly measurable by a single sensor

Virtual IoT

- Virtualised instance
- Functionality
 - Data collection
 - Data validation
 - Metadata annotation
 - Processing
 - Actuation
- Configurations
- Deployment
 - Cloud
 - Edge





S. Madria, V. Kumar, R. Dalvi. (2014) Sensor Cloud: A Cloud of Virtual Sensors. *IEEE Software*, 31(2):70-77, IEEE.

Components

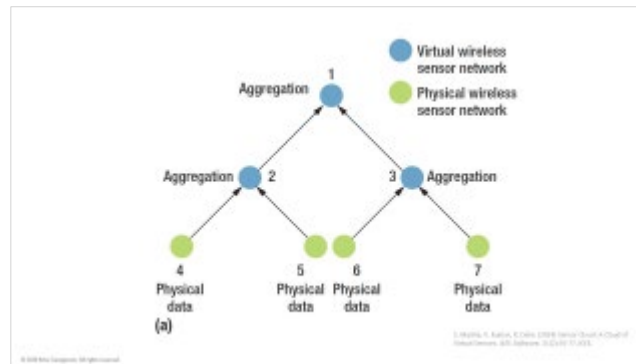
Processing

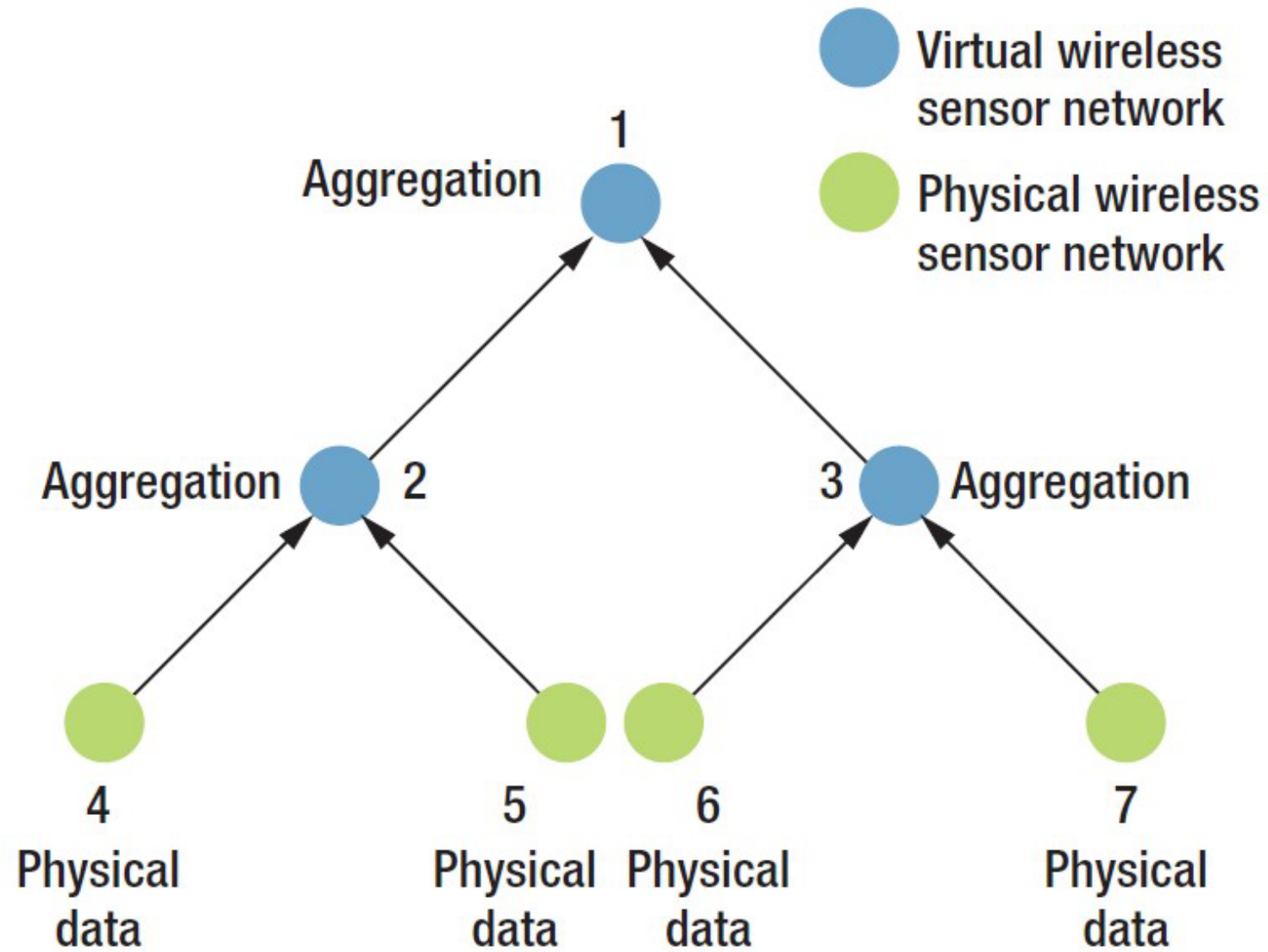
Configuration

Sensors and actuators gateway

Operations

- Accumulation
- Selection
- Aggregation
- Qualification
- Prediction
- Computation





(a)

S. Madria, V. Kumar, R. Dalvi. (2014) Sensor Cloud: A Cloud of Virtual Sensors. *IEEE Software*, 31(2):70-77, IEEE.

Example

A smart city with connected vehicles that can report data about environment conditions and receive actions or notifications. The smart city would like to examine the road conditions and inform or instruct connected vehicles about the possible actions to take in certain road conditions.

IOT DATA

Telemetry

- Message
 - Type ID
 - Instance ID
 - Timestamp
 - Value

- JSON format

```
{  
  "type-id": <string>,  
  "instance-id": <string>,  
  "timestamp": <ISO8601>,  
  "value": <any>  
}
```

Telemetry

- Example

```
{  
  "type-id" : "de.uni-stuttgart.iaas.sc.grovepi/light",  
  "instance-id" : "0xe238dbw3c277",  
  "timestamp" : "2018-05-02T07:20:30.262Z",  
  "value" : {  
    "lux" : "1142.0"  
  }  
}
```



```
import time
from datetime import datetime
import json
from simulatediot import TemperatureSensor
```

```
class Simulator:
```

```
    def __init__(self, interval):
        self.interval = interval
```

```
    def start(self):
```

```
        ts = TemperatureSensor(20, 10, 16, 30)
```

```
        while True:
```

```
            dt = datetime.now().strftime("%d-%m-%YT%H:%M:%S")
```

```
            message = {
```

```
                "type-id": "de.uni-stuttgart.iaas.sc." + ts.sensor_type,
```

```
                "instance-id": ts.instance_id,
```

```
                "timestamp": dt,
```

```
                "value": {
```

```
                    ts.unit: ts.sense()
```

```
                }
```

```
            }
```

```
            jmsg = json.dumps(message, indent=4)
```

```
            print(jmsg)
```

```
            time.sleep(self.interval)
```

```
s = Simulator(5)
```

```
s.start()
```

